

Beyond This Course

A Roadmap for Deep Learning in Motor Control

Self-contained reference with code examples for independent study

Distributed with the Week 15 lecture — not examinable

Architecture	What it does	Motor control use case
RNN / LSTM	Processes sequences step by step; remembers temporal context	Decode reach direction from the full neural time-series (not just mean rates)
CNN (1D)	Detects local spatial or spectral patterns via learned filters	Extract features from raw EMG waveforms; detect motor unit action potentials
Autoencoder	Compresses data to a bottleneck, then reconstructs; learns nonlinear manifold	Nonlinear dimensionality reduction of neural population activity (beyond PCA)
Transformer	Attends to all time steps simultaneously; captures long-range dependencies	Population-level modelling of many neurons across an entire trial

1. Recurrent Neural Networks and LSTMs

Processing neural time-series without collapsing them to means

In Weeks 5–13, we averaged each neuron’s activity over the entire trial to produce a single mean rate. Week 14 showed that keeping the temporal structure (80 neurons $\times$ 50 time bins = 4,000 features) dramatically improved impairment detection. But feeding all 4,000 features as a flat vector to an MLP ignores the sequential nature of the data — the network does not know that time bin 5 comes after time bin 4.

A Recurrent Neural Network (RNN) processes the time-series one step at a time, maintaining a hidden state that carries information from previous steps. At each time step t , it receives the current input (firing rates of all 80 neurons at time t) and the previous hidden state, and produces a new hidden state. After processing all 50 time steps, the final hidden state summarises the entire temporal sequence and is fed to a classifier.

The vanilla RNN suffers from the vanishing gradient problem: gradients shrink exponentially as they are backpropagated through many time steps, making it difficult to learn long-range dependencies. The Long Short-Term Memory (LSTM) network solves this with a gating mechanism: three gates (input, forget, output) control what information is stored, discarded, or passed forward. This allows the LSTM to remember the onset timing from the start of a trial while processing activity 400 ms later.

PyTorch example: LSTM for direction decoding

```
import torch
import torch.nn as nn

class LSTMDecoder(nn.Module):
    def __init__(self, n_neurons=80, hidden=64, n_classes=8):
        super().__init__()
        self.lstm = nn.LSTM(n_neurons, hidden, batch_first=True)
        self.fc = nn.Linear(hidden, n_classes)

    def forward(self, x):
        # x shape: (batch, 50 time_steps, 80 neurons)
        _, (h_n, _) = self.lstm(x)      # h_n: (1, batch, 64)
        return self.fc(h_n.squeeze(0))  # (batch, 8)

# Usage:
# X_temporal reshaped to (480, 50, 80) instead of (480, 4000)
# model = LSTMDecoder()
# out = model(torch.tensor(X_resaped, dtype=torch.float32))
```

The key difference from the Week 14 MLP: the input is shaped as (batch, 50, 80) — a sequence of 50 time steps, each with 80 neuron rates — rather than a flat 4,000-element vector. The LSTM processes this sequence in order, building a hidden representation that respects temporal structure.

When to use RNNs/LSTMs

- Your data is inherently sequential (time-series of neural activity, EMG, kinematics).
- The signal of interest is in the temporal dynamics (onset timing, ramping, oscillations) rather than average rates.

- You have enough data ($>1,000$ trials) for the LSTM's additional parameters to be worth the complexity.

Key references

- Hochreiter S, Schmidhuber J. Long short-term memory. *Neural Computation*, 9(8), 1735–1780, 1997.
- Pandarinath C et al. Inferring single-trial neural population dynamics using sequential auto-encoders. *Nature Methods*, 15, 805–815, 2018. (LFADS — a variational RNN for neural data.)

2. Convolutional Neural Networks (1D)

Detecting local patterns in neural and EMG signals

A 1D CNN slides small learned filters across the time dimension of a signal, detecting local patterns like bursts, oscillations, or motor unit action potentials. Unlike the LSTM, which processes sequentially, the CNN processes all positions in parallel and is sensitive to local structure regardless of when it occurs in the trial.

Each convolutional filter learns a template — a short pattern (e.g., 5 time bins wide) that, when convolved with the input, produces a high activation wherever the pattern appears. Multiple filters detect multiple patterns. A pooling layer then summarises across time (e.g., max-pooling takes the strongest activation regardless of position), making the representation invariant to small timing shifts.

For motor control data, 1D CNNs are particularly useful for raw EMG processing: detecting motor unit action potential shapes, extracting frequency-domain features, and identifying burst patterns — tasks where the relevant signal is a local waveform shape rather than a global temporal trajectory.

PyTorch example: 1D CNN for EMG classification

```
class CNN1D(nn.Module):
    def __init__(self, n_channels=6, n_classes=8):
        super().__init__()
        self.conv1 = nn.Conv1d(n_channels, 32, kernel_size=5, padding=2)
        self.conv2 = nn.Conv1d(32, 64, kernel_size=5, padding=2)
        self.pool = nn.AdaptiveMaxPool1d(1) # global max pool
        self.fc = nn.Linear(64, n_classes)

    def forward(self, x):
        # x shape: (batch, 6 muscles, T time_samples)
        x = torch.relu(self.conv1(x)) # (batch, 32, T)
        x = torch.relu(self.conv2(x)) # (batch, 64, T)
        x = self.pool(x).squeeze(-1) # (batch, 64)
        return self.fc(x) # (batch, 8)
```

The CNN treats each muscle as a channel (like RGB in an image CNN) and learns filters that detect patterns across time. Two convolutional layers build a hierarchy: the first detects simple patterns (individual muscle activations), the second detects combinations (co-contractions).

When to use 1D CNNs

- Your signal contains local patterns (waveforms, oscillation bursts, motor unit potentials) rather than global temporal dynamics.
- You want translation invariance: the same burst should be detected whether it occurs at 100 ms or 200 ms.
- You have multi-channel signals (EMG electrodes, EEG channels, neural arrays) where spatial patterns matter.

Key references

- LeCun Y, Bengio Y, Hinton G. Deep learning. *Nature*, 521, 436–444, 2015.
- Cote-Allard U et al. Deep learning for electromyographic hand gesture signal classification. *IEEE TNSRE*, 27(4), 760–771, 2019.

3. Autoencoders

Nonlinear dimensionality reduction beyond PCA

PCA (Week 4) found linear projections that captured maximum variance. But neural population activity often lies on a curved manifold — a nonlinear surface in high-dimensional space. An autoencoder learns this manifold by training a network to compress the data through a bottleneck and then reconstruct it.

The encoder maps the 80-dimensional firing rate vector to a low-dimensional latent space (e.g., 3 dimensions). The decoder maps it back to 80 dimensions. The network is trained to minimise reconstruction error: the output should match the input. After training, the bottleneck activations are the learned low-dimensional representation — like PCA scores, but capturing nonlinear structure.

A variational autoencoder (VAE) adds a probabilistic twist: the latent space is regularised to follow a Gaussian distribution, producing smooth, continuous manifolds. This is the basis of LFADS (Latent Factor Analysis via Dynamical Systems), one of the most influential tools in computational neuroscience for extracting neural dynamics.

PyTorch example: autoencoder for neural data

```
class Autoencoder(nn.Module):
    def __init__(self, input_dim=80, latent_dim=3):
        super().__init__()
        self.encoder = nn.Sequential(
            nn.Linear(input_dim, 64), nn.ReLU(),
            nn.Linear(64, 32), nn.ReLU(),
            nn.Linear(32, latent_dim) # bottleneck
        )
        self.decoder = nn.Sequential(
            nn.Linear(latent_dim, 32), nn.ReLU(),
            nn.Linear(32, 64), nn.ReLU(),
            nn.Linear(64, input_dim) # reconstruct
        )

    def forward(self, x):
        z = self.encoder(x) # (batch, 3)
        x_hat = self.decoder(z) # (batch, 80)
        return x_hat, z

# Loss: MSE between input and reconstruction
# loss = F.mse_loss(x_hat, x)
# After training, z = model.encoder(X) gives latent coords
```

Compare with PCA: PCA's latent space is a linear projection (80D → 3D via matrix multiplication). The autoencoder's latent space is a nonlinear projection (80D → 64 → 32 → 3D via multiple relu transformations). If the data lies on a curved manifold, the autoencoder captures structure that PCA misses.

When to use autoencoders

- You suspect the data lies on a nonlinear manifold (e.g., rotational dynamics in motor cortex).
- You want unsupervised dimensionality reduction without labels.

- You plan to use the latent representation for downstream tasks (clustering, visualisation, dynamics modelling).

Key references

- Pandarinath C et al. Inferring single-trial neural population dynamics using sequential auto-encoders. Nature Methods, 15, 805–815, 2018.
- Kingma DP, Welling M. Auto-encoding variational Bayes. ICLR, 2014.

4. Transformers

Population-level modelling with attention

The transformer architecture, originally developed for natural language processing, has emerged as a powerful tool for neural data. Its key innovation is the self-attention mechanism: instead of processing the sequence step by step (like an LSTM), the transformer compares every time step with every other time step simultaneously. This allows it to capture long-range dependencies — for example, that a preparatory burst at time 0 predicts a specific movement pattern at time 400 ms — without the vanishing gradient problem.

For neural population data, the transformer can attend to both temporal and neuronal dimensions. A trial with 80 neurons $\times$ 50 time bins can be treated as a sequence of 50 tokens (each token = 80 neuron rates at one time step), or as a sequence of 80 tokens (each token = 50 time bins for one neuron), or even as 4,000 individual neuron-time tokens. The attention weights reveal which neuron-time combinations the model considers important — a form of learned interpretability.

PyTorch example: transformer encoder for neural data

```
class NeuralTransformer(nn.Module):
    def __init__(self, n_neurons=80, n_bins=50,
                  d_model=64, nhead=4, n_layers=2, n_classes=8):
        super().__init__()
        self.input_proj = nn.Linear(n_neurons, d_model)
        self.pos_enc = nn.Parameter(
            torch.randn(1, n_bins, d_model) * 0.02)
        encoder_layer = nn.TransformerEncoderLayer(
            d_model=d_model, nhead=nhead, batch_first=True)
        self.transformer = nn.TransformerEncoder(
            encoder_layer, num_layers=n_layers)
        self.fc = nn.Linear(d_model, n_classes)

    def forward(self, x):
        # x: (batch, 50 time_steps, 80 neurons)
        x = self.input_proj(x) + self.pos_enc # (batch, 50, 64)
        x = self.transformer(x)               # (batch, 50, 64)
        x = x.mean(dim=1)                     # average over time steps
        return self.fc(x)                     # (batch, 8)
```

The positional encoding (pos_enc) tells the transformer about time order — without it, the model treats the sequence as an unordered set. The multi-head attention (nhead=4) allows different attention heads to focus on different aspects of the temporal dynamics.

When to use transformers

- You have large datasets (>5,000 trials). Transformers have many parameters and require substantial data to train effectively.
- You want to model interactions across both neurons and time simultaneously.
- You want interpretable attention weights that reveal which time steps and neurons drive the prediction.

Key references

- Vaswani A et al. Attention is all you need. NeurIPS, 2017.

- Ye J, Bhatt D, Bhatt R, Pandarinath C. Neural data transformer 2: multi-context pretraining for neural spiking activity. NeurIPS, 2023.
- Azabou M et al. A unified, scalable framework for neural population decoding. NeurIPS, 2023.

Summary: Choosing a Deep Learning Architecture

The table below maps research scenarios to recommended architectures. Each row connects a data type, a neuroscience question, and a starting architecture with its PyTorch building block.

Data Type	Question	Architecture	PyTorch Class
Neural time-series (80 × 50 bins)	Decode direction from temporal dynamics	LSTM	nn.LSTM
Raw EMG waveforms (6 ch × 1000 samples)	Classify gestures from waveform shape	1D CNN	nn.Conv1d
Population rates (80 neurons)	Find nonlinear neural manifold	Autoencoder / VAE	nn.Linear (enc+dec)
Multi-neuron × time (large dataset)	Model population dynamics with long-range deps	Transformer	nn.TransformerEncoder
Any small dataset (<500 trials)	Anything	LR, LDA, or RF (Weeks 5-14!)	sklearn

The last row is the most important. On small datasets with well-designed features — the situation most graduate students actually face — the classical methods from this course remain the right choice. Deep learning earns its keep when data is abundant and features are not pre-designed. The decision framework from the Week 15 lecture (Figure 5) remains your primary guide; this handout extends it for the cases where the answer is ‘MLP or deeper.’

Further reading

- Goodfellow I, Bengio Y, Courville A. Deep Learning. MIT Press, 2016. Free online: deeplearningbook.org
- PyTorch tutorials: pytorch.org/tutorials — start with ‘Sequence Models and LSTM’ and ‘Transfer Learning Tutorial.’
- Neuromatch Academy Deep Learning course: deeplearning.neuromatch.io — free, neuroscience-focused.
- Glaser JI et al. The roles of supervised ML in systems neuroscience. Progress in Neurobiology, 175, 2019.